CSSS 441 Software Security I: End-of-Semester Projects and Weekly Practicals

Oct 25, 2026 · @Developer

How the projects work

Each group of 3 to 5 students picks one of the ten projects below by Week 2, builds it through the semester, and demonstrates it in Week 10. Every project is a working system the group designs, implements, secures, tests and defends, so each member leaves with a public portfolio piece and a resume line.

Selection. Each project can be taken by at most one group. Groups submit three ranked choices in Week 1 and allocation is first come, first served. A group may propose its own topic if it matches the depth of the list and is approved by Week 2.

Rules of engagement. Build and test only on machines, containers, cloud accounts and domains the group owns or the course provides. Use public datasets and your own generated traffic. Never scan, probe or test a third-party system; doing so may be an offence under the Cybersecurity Act, 2020 (Act 1038). Any real bug found in someone else's software is reported through the lecturer, following that project's disclosure policy.

Milestones.

Week	Milestone	Marks
2	Topic confirmed; repository created with README and team roles	0
4	Threat model and architecture: DFD, STRIDE table, chosen controls	5%
7	Working core demo to the lecturer; feedback only	0
10	Final repository, report, demo video and live presentation	25%

What every group submits in Week 10.

- A Git repository with a README, one-command setup (Docker Compose, Ansible or Terraform), CI and tests.
- The updated threat model and an architecture diagram.

- A security test suite plus evidence: test output, scan reports, packet captures, screenshots.
- A report of 10 to 15 pages: design, controls, verification, residual risk, individual contributions.
- A 5-minute demo video and a 15-minute live presentation with 5 minutes of questions.
- A one-page portfolio summary per student: what they built, the tools used, and two resume bullets.

Final marking (25% of the course).

Criterion	Marks	What earns full marks
Working implementation	30	Every core feature works from a clean checkout with one command
Security engineering	25	Controls trace back to the threat model; no homemade crypto
Verification	20	Automated tests and evidence prove each security claim
Documentation	10	A stranger can deploy, use and audit the system from the docs
Demo and defence	15	Every member explains their part and answers questions accurately

Individual marks may be moderated using Git history and peer evaluation.

Project 1: Secure mobile-money payments API

The group builds the backend of a mobile-money-style wallet service and proves it meets OWASP ASVS 5.0 Level 2 for authentication, sessions, access control, validation and logging. Course links: Weeks 2, 4 and 6.

You will build

1. Registration, login and account recovery with Argon2id password hashing, TOTP and passkey (WebAuthn) second factors, and per-account and per-IP throttling.
2. Wallets, balances and transfers on a double-entry ledger in PostgreSQL, with database transactions, daily limits and idempotency keys so a retried request never pays twice.
3. A deny-by-default authorization layer: role checks (customer, merchant, agent, admin) plus an ownership check on every wallet and transaction route.

4. Merchant webhooks signed with HMAC-SHA-256 and a timestamp, verified in constant time, with replay protection.
5. A tamper-evident audit log in which each entry carries the hash of the previous one, plus a verifier that detects edits.
6. Strict request schemas, output encoding, security headers and generic error messages.
7. An admin console protected by MFA and reachable only from an allow-listed network.

Security bar. Every ASVS requirement the group claims is mapped to a test or a piece of evidence. No secret sits in source code; keys come from environment secrets or HashiCorp Vault.

Stack and lab. FastAPI or NestJS, PostgreSQL, Redis, Docker Compose; pytest or Jest; Semgrep and OSV-Scanner in CI; an OWASP ZAP scan against the group's own staging deployment.

Evidence to submit. ASVS checklist with pass or fail and a link to proof for each item; test report; ZAP report before and after fixes; a demo that replays a webhook and shows it rejected.

Resume line. *Built a mobile-money payments API with passkey MFA, an idempotent double-entry ledger and HMAC-signed webhooks, verified against OWASP ASVS 5.0 Level 2 with an automated security test suite.*

Project 2: DevSecOps pipeline with security gates

The group builds a CI/CD pipeline that scans, signs and gates every change to a real web application, and shows it blocking a vulnerable or unsigned build. Course links: Weeks 2, 3 and 9.

You will build

1. Pre-commit hooks for secret scanning (gitleaks) and linting, mirrored in CI so they cannot be skipped.
2. Static analysis with Semgrep and CodeQL, including at least five custom Semgrep rules written for the target app's own risky patterns.
3. Dependency and container scanning with OSV-Scanner and Trivy, and a CycloneDX SBOM from Syft on every build.
4. A hardened container image: non-root user, minimal base image, read-only filesystem, no shell where possible.
5. Infrastructure-as-code scanning with Checkov over the Docker Compose or Kubernetes files.

6. An ephemeral staging deployment in the pipeline, scanned with an OWASP ZAP baseline scan.
7. Keyless image signing with Sigstore cosign and SLSA provenance, verified before deployment.
8. A gate that fails the build on critical findings, with results collected in DefectDojo.

Security bar. A commit containing a planted secret, a known-vulnerable dependency or an unsigned image must be stopped automatically, and the pipeline must say why.

Stack and lab. GitHub Actions or GitLab CI; Docker; a local k3s or kind cluster; the target app is the group's own or an open-source app such as OWASP Juice Shop.

Evidence to submit. Pipeline definitions; three blocked-build demonstrations; findings before and after remediation; mean time from finding to fix; the custom rules with their test cases.

Resume line. *Built a DevSecOps pipeline (Semgrep, CodeQL, Trivy, Syft, ZAP, cosign) that generates SBOMs, signs images and blocks vulnerable or unsigned builds before deployment.*

Project 3: SOC-in-a-box network detection and monitoring

The group deploys and tunes a security monitoring stack for a small lab network, then measures how well its detections work on public datasets. Course links: Weeks 2 and 8.

You will build

1. A lab network of three to five hosts (web server, database, workstation, gateway) in VMs or containers, deployed by Ansible or Docker Compose.
2. Network monitoring with Suricata and Zeek on a mirrored interface, and host monitoring with Wazuh agents.
3. Log collection into Wazuh or OpenSearch: firewall (nftables), web server, SSH and authentication logs, Zeek connection, DNS and HTTP logs.
4. At least 15 custom detection rules (Suricata and Sigma or Wazuh), each mapped to a MITRE ATT&CK technique and each with a test case.
5. Dashboards for alert volume by severity, failed logins, top talkers and DNS anomalies.
6. A validation harness that replays public captures (for example CIC-IDS2017) and a clean baseline, then reports detection and false-positive rates per rule.
7. Five incident-response playbooks and one recorded tabletop exercise.

Security bar. Every rule must fire on its test traffic and stay quiet on the baseline, or the report must explain the trade-off.

Stack and lab. Suricata, Zeek, Wazuh or OpenSearch, nftables, tcpreplay, Ansible; 16 GB RAM recommended, or a cloud lab within free credits.

Evidence to submit. ATT&CK coverage matrix; per-rule precision and false-positive table; dashboards; playbooks; the deployment scripts.

Resume line. *Deployed and tuned a SOC stack (Suricata, Zeek, Wazuh) with 15+ ATT&CK-mapped detections, validated on CIC-IDS2017 with measured false-positive rates.*

Project 4: Zero-trust remote access gateway

The group replaces "trust the internal network" with per-request identity, device and certificate checks for a set of internal lab applications. Course links: Weeks 4, 7 and 9.

You will build

1. A private certificate authority with an offline root and an online intermediate (smallstep step-ca), issuing certificates that live for 24 hours or less.
2. Single sign-on with Keycloak (OIDC) and MFA by TOTP or passkey.
3. An identity-aware proxy (Pomerium, or oauth2-proxy with NGINX) in front of three internal apps, with per-route policies by group and device state.
4. Mutual TLS between every internal service, with automatic certificate renewal.
5. A WireGuard tunnel for administrator access to the hosts themselves.
6. A small device-posture agent that reports OS version, patch age and disk-encryption status, and a gateway policy that denies non-compliant devices.
7. Host hardening against a CIS benchmark, scored with Lynis or OpenSCAP before and after.
8. Central access logs and an automatically generated monthly access review.

Security bar. A stolen password alone, an expired certificate, a revoked device or a direct connection that bypasses the proxy must all be refused, and each refusal must appear in the logs.

Stack and lab. step-ca, Keycloak, Pomerium or oauth2-proxy, NGINX, WireGuard, Lynis or OpenSCAP, Docker Compose or k3s.

Evidence to submit. Architecture diagram with trust boundaries; policy files; captures showing mTLS and tunnel encryption; hardening scores; a test matrix of allowed and denied access attempts.

Resume line. *Designed a zero-trust access gateway (Keycloak SSO, Pomerium, step-ca mTLS, WireGuard) enforcing identity, device-posture and short-lived certificate checks on every request.*

Project 5: End-to-end encrypted messaging app

The group builds a web or Android chat app in which the server only ever stores ciphertext, using a vetted protocol library rather than its own cryptography. Course links: Weeks 5, 6 and 7.

You will build

1. One-to-one messaging on the Signal Protocol through libsignal (X3DH or PQXDH key agreement plus the Double Ratchet), or Olm through vodozemac.
2. A key server that holds only public identity keys and signed prekeys, with prekey replenishment.
3. Key verification by safety number or QR code, and a visible warning when a contact's identity key changes.
4. Encrypted attachments: a random key per file, AES-256-GCM, with the key sent inside the encrypted message.
5. Encrypted message storage on the device: SQLCipher for the local database, with its key held in the Android Keystore or the browser's non-extractable Web Crypto keys.
6. Disappearing messages and remote session reset.
7. Group chat with sender keys (or Megolm) as the stretch goal.

Security bar. A full dump of the server database and its traffic must reveal no message content. The report shows why a key compromised today cannot decrypt last week's messages.

Stack and lab. libsignal or vodozemac; Node.js or Go server; React or Kotlin client; PostgreSQL; WebSockets over TLS 1.3.

Evidence to submit. A cryptographic design document with message-flow diagrams; the database dump; a capture of server traffic; tests for key-change warnings and tampered ciphertext; a threat model that names what the server can still learn (metadata).

Resume line. *Built an end-to-end encrypted messenger on the Signal Protocol (libsignal) with key verification, encrypted attachments and forward secrecy; the server stores ciphertext only.*

Project 6: Zero-knowledge password manager

The group builds a password manager with a sync server that never learns the master password or any stored secret. Course links: Weeks 4, 5 and 6.

You will build

1. A client (browser extension or desktop app) that derives keys from the master password with Argon2id, using parameters the group justifies with a timing benchmark.
2. Vault encryption with XChaCha20-Poly1305 or AES-256-GCM through libsodium or the `cryptography` library, with a separate key per vault item.
3. Login to the sync server with OPAQUE (for example the opaque-ke library) or a separately derived authentication key, so the server never receives the master password.
4. Sharing an item with another user by wrapping its key to their X25519 public key.
5. A password generator and a breached-password check against the Have I Been Pwned range API (k-anonymity: only a 5-character hash prefix leaves the device).
6. Autofill that checks the exact origin before filling, so a look-alike domain gets nothing.
7. TOTP code storage, encrypted export, and an emergency-access flow with a waiting period.

Security bar. A copy of the server database plus all server logs must not reveal any password or vault item. A single flipped bit in a stored vault must be detected and refused.

Stack and lab. TypeScript (WebExtension) or Electron or Kotlin client; Rust, Go or Node.js server; libsodium, argon2, opaque-ke; PostgreSQL.

Evidence to submit. Key-hierarchy diagram; Argon2id benchmark and the parameter choice; tamper and wrong-origin tests; a server-compromise analysis.

Resume line. *Built a zero-knowledge password manager (Argon2id, XChaCha20-Poly1305, OPAQUE login, X25519 sharing) whose server never sees the master password or vault contents.*

Project 7: Private PKI and certificate lifecycle manager

The group runs a two-tier certificate authority for its lab and builds a dashboard that issues, renews, revokes and monitors every certificate automatically. Course links: Weeks 6, 7 and 9.

You will build

1. An offline root CA and an online intermediate with documented key ceremonies, name constraints and path-length limits.
2. An ACME endpoint (step-ca) and automatic issuance and renewal for at least five lab services using certbot or lego, with no downtime during rotation.
3. Revocation through a CRL and an OCSP responder, and a client test proving a revoked certificate is refused.

4. An inventory service that connects to the group's own TLS endpoints and records issuer, expiry, key type and size, SANs and chain completeness.
5. A web dashboard with expiry alerts at 30, 7 and 1 days, and email or Telegram notifications.
6. Certificate Transparency monitoring for a domain the group owns, through the crt.sh API, with an alert on any unexpected issuance.
7. TLS configuration grading with testssl.sh, and a policy check that flags anything below TLS 1.2 with AEAD suites.

Security bar. Certificates last 30 days or less and renew themselves. A wrong hostname, an expired certificate, a missing intermediate or a revoked certificate is each detected and reported within one scan cycle.

Stack and lab. step-ca or OpenSSL, certbot or lego, NGINX, Python (FastAPI) or Go, PostgreSQL, testssl.sh; one inexpensive public domain for the CT part (optional).

Evidence to submit. CA hierarchy diagram and ceremony notes; renewal logs over two weeks; revocation and failure-injection tests; testssl.sh results before and after.

Resume line. *Built a private PKI with automated ACME issuance, OCSP revocation and Certificate Transparency monitoring, plus a dashboard that inventories and grades every TLS endpoint.*

Project 8: Software supply-chain integrity platform

The group builds a tool that tells a team which of its dependencies are vulnerable, which of those actually matter, and whether a build can be trusted, then enforces that decision in CI. Course links: Weeks 6 and 9.

You will build

1. SBOM generation for Python and JavaScript repositories in CycloneDX (Syft or cdxgen).
2. Vulnerability matching against OSV, enriched with EPSS scores and the CISA KEV list.
3. A reachability check that parses the repository's imports and call sites and marks each vulnerable package as used or unused.
4. VEX output (CycloneDX VEX or OpenVEX) recording why each finding is or is not exploitable in this product.
5. Signing of build artefacts and attestations with cosign and in-toto, and verification of SLSA provenance.
6. A typosquatting check that compares new dependency names against popular package names by edit distance.

7. A policy engine in OPA (Rego) that allows or blocks a deployment, wired into a GitHub Action.
8. A dashboard that ranks risk across repositories.

Security bar. Run the tool on three real open-source repositories and report how many findings the reachability and VEX steps correctly set aside, checked by hand on a sample of at least 20.

Stack and lab. Python or Go; Syft or cdxgen; OSV API; EPSS and KEV feeds; cosign, in-toto; OPA; GitHub Actions; PostgreSQL and React.

Evidence to submit. SBOMs and VEX files for the three repositories; the hand-checked accuracy sample; policy decision logs; a blocked unsigned build.

Resume line. *Built a supply-chain security platform that generates SBOMs, prioritises CVEs with EPSS, KEV and reachability, emits VEX, and enforces signed SLSA provenance through OPA policies in CI.*

Project 9: Phishing and smishing detection service

The group builds a service that lets ordinary users check a suspicious link or SMS (including mobile-money fraud messages) and get a clear verdict, then measures how accurate it is. Course links: Weeks 2 and 8.

You will build

1. A labelled dataset from public sources (PhishTank or OpenPhish URL feeds, the UCI SMS Spam Collection), plus scam SMS donated with written consent and stripped of names and numbers.
2. URL features: lexical patterns, domain age from RDAP, certificate issuer and age, redirect chain, and similarity to known brand names.
3. A link checker that fetches pages only inside an isolated, disposable container with no access to the lab network.
4. Two classifiers, a transparent rule set and a machine-learning model (for example gradient boosting on URL features and a fine-tuned small text model for SMS), compared on the same held-out test set.
5. A REST API, a Telegram or WhatsApp bot that answers "is this safe?", and a browser extension that warns before a flagged page loads.
6. An analyst dashboard with user feedback, false-positive review and a monthly model-drift report.
7. Privacy by design under the Data Protection Act, 2012 (Act 843): no message stored beyond what the verdict needs, and a retention limit.

Security bar. Report precision, recall and false-positive rate on a test set drawn from a later time period than the training set, and explain each misclassification category.

Stack and lab. Python (scikit-learn, Hugging Face Transformers), FastAPI, PostgreSQL, Docker; Telegram Bot API; a WebExtension.

Evidence to submit. Data card and consent form; evaluation report with confusion matrices; latency figures; the privacy impact assessment; a live demo through the bot.

Resume line. *Built a phishing and smishing detection service (ML plus rules, REST API, Telegram bot, browser extension) evaluated on time-split data, with a privacy impact assessment under Ghana's Act 843.*

Project 10: Continuous fuzzing and a memory-safe rewrite

The group sets up continuous fuzzing for a small C parsing library, fixes what it finds, and replaces the parsing core with a Rust version proven equivalent by differential fuzzing. Course links: Weeks 3 and 9.

You will build

1. A fork of a small open-source C parser (JSON, INI, CSV, URL or an image-header reader) chosen with the lecturer, or the course-provided parser.
2. Fuzz harnesses for libFuzzer and AFL++, with seed corpora, dictionaries, and AddressSanitizer and UndefinedBehaviorSanitizer enabled.
3. Coverage measurement with llvm-cov, tracked weekly, and new harnesses for the code the fuzzers never reach.
4. Continuous fuzzing in CI with ClusterFuzzLite, running on every pull request and on a nightly schedule for at least two weeks.
5. A triage workflow: deduplicate crashes, minimise inputs, find the root cause, assign a CWE, fix it and add a regression test.
6. A Rust rewrite of the parsing core behind the same C interface (via FFI), plus a differential fuzzer that feeds both versions the same input and flags any difference.
7. Benchmarks of speed and memory for the C and Rust versions.

Security bar. Every crash found is either fixed with a regression test or documented as a known issue, and the Rust version survives the differential fuzzer with no mismatches over a full night.

Stack and lab. clang, libFuzzer, AFL++, ASan, UBSan, llvm-cov, ClusterFuzzLite, Rust (cargo-fuzz), GitHub Actions. Real bugs in upstream code are reported through the lecturer, following the project's disclosure policy.

Evidence to submit. Coverage-over-time chart; crash and fix log with CWE mapping; differential-fuzzing results; benchmark table; links to any upstream reports or merged fixes.

Resume line. *Set up continuous fuzzing (libFuzzer, AFL++, ClusterFuzzLite) for a C parser, fixed the memory-safety bugs it found, and shipped a differentially fuzzed Rust replacement.*

Weekly practical exercises

The ten practicals start in the last 30 to 45 minutes of each class and are finished at home. They sit beside the graded labs: each one builds a small, reusable component or habit that groups carry into their project. Students work on their own machines, on their own code and lab VMs only.

Week 1: Threat-model your project

Goal. Turn the chosen project into a first threat model.

1. Draw a level-1 data flow diagram of the project in OWASP Threat Dragon, with every trust boundary marked.
2. Apply STRIDE-per-element and list at least 12 threats.
3. Rate each threat with the course risk matrix and give one reason per rating.
4. Pick the top five threats and name the control that will address each.

Submit. The Threat Dragon JSON file and a one-page summary. **Done when** every data flow that crosses a boundary has at least one threat recorded against it.

Week 2: An input-handling kit

Goal. Build the validation and data-access layer the project will reuse.

1. Define request schemas with Pydantic (Python) or Zod (TypeScript): types, lengths, formats and ranges.
2. Write a data-access module that uses parameterised queries only.
3. Render all user data through an auto-escaping template engine or framework.
4. Write at least 20 negative unit tests drawn from the OWASP cheat sheets (quotes, very long strings, Unicode tricks, wrong types).
5. Run Semgrep and fix every finding.

Submit. The module, the tests and the Semgrep output. **Done when** all tests pass and Semgrep reports nothing.

Week 3: Sanitizer drill

Goal. Make memory errors loud, then remove them.

1. Compile the three provided C exercises with `-Wall -Wextra -fsanitize=address,undefined`.
2. Fix every warning and every sanitizer report at its root cause.
3. Add boundary tests for empty input, maximum length, maximum length plus one and a very large input.
4. Port one of the functions to Rust and compare the two versions in five lines.

Submit. The diffs, the clean build log and the test output. **Done when** the builds are warning-free and the sanitizers stay silent on every test.

Week 4: An authentication module

Goal. Build login the project can drop in.

1. Store passwords with Argon2id and rehash old hashes at the next login.
2. Add TOTP enrolment and verification (pyotp or otplib).
3. Issue session cookies with `Secure`, `HttpOnly` and `SameSite`, and a new session ID at login.
4. Throttle login per account and per IP address.
5. Write tests for each of the four points above.

Submit. The module and the test report. **Done when** the tests prove throttling, cookie flags and session rotation.

Week 5: A file-encryption utility

Goal. Encrypt data correctly with a vetted library.

1. Build a command-line tool that encrypts and decrypts files with AES-256-GCM from the `cryptography` library, using a fresh 96-bit nonce per file.
2. Read the key from an environment secret or HashiCorp Vault, never from source code.
3. Add a hybrid mode that encrypts to a recipient's public key (libsodium sealed boxes with X25519, or RSA-OAEP).
4. Test that a single changed byte in the ciphertext makes decryption fail.

Submit. The tool, its tests and a short note on nonce handling. **Done when** the tamper test fails closed and Bandit reports no cryptography findings.

Week 6: An integrity toolkit

Goal. Detect changes and prove who released a file.

1. Build a file-integrity monitor that records SHA-256 hashes of a directory and protects the record with HMAC-SHA-256.
2. Report added, removed and modified files on each run.
3. Sign a release archive with Ed25519 and write a verifier that refuses a tampered archive or an unknown key.
4. Compare tags and signatures only with constant-time functions.

Submit. The tool, a sample report and the verifier tests. **Done when** every tampering case is detected and the verifier fails closed.

Week 7: TLS and SSH hardening

Goal. Serve the project over correctly configured TLS and lock down its host.

1. Create a private CA with OpenSSL or step-ca and issue a certificate for the project's lab hostname.
2. Configure NGINX for TLS 1.3 (TLS 1.2 with AEAD suites only as a fallback) and add HSTS.
3. Scan the server with testssl.sh and fix every high or medium finding.
4. Run ssh-audit on the host, switch to Ed25519 key-only login and disable root login.

Submit. The configurations and before-and-after scan reports. **Done when** testssl.sh and ssh-audit show no high findings.

Week 8: Firewall and detection

Goal. Allow only what the project needs, and see when something else happens.

1. Write a default-deny nftables policy for the project server that opens only its required ports.
2. Install Suricata on the lab VM and write three custom rules for the project (for example repeated failed logins, a port sweep, an unusual DNS pattern).
3. Trigger each rule from a second lab VM that the group controls, then record a normal-traffic baseline.
4. Tune any rule that fires on the baseline.

Submit. The ruleset, the policy and alert screenshots. **Done when** each rule fires on its test and stays quiet on the baseline.

Week 9: VPN and dependency hygiene

Goal. Protect administrator traffic and clean up the project's dependencies.

1. Build a WireGuard tunnel between two lab VMs for administrator access.
2. Capture traffic on the physical interface and on the tunnel interface, and explain the difference.
3. Generate a CycloneDX SBOM of the project with Syft and scan it with Grype or OSV-Scanner.
4. Upgrade or replace every dependency with a critical or high finding, or record why it is not exploitable here.

Submit. The WireGuard configuration with keys removed, both captures, the SBOM and the scan before and after. **Done when** the tunnel works and no unexplained critical findings remain.

Week 10: Peer security review

Goal. Practise finding and reporting issues the way industry teams do.

1. Swap repositories with another group.
2. Work through a checklist covering input handling, authentication, access control, cryptography, configuration and dependencies.
3. Write five findings, each with a title, severity, location, evidence and a suggested fix.
4. The receiving group accepts or rebuts each finding in writing before its presentation.

Submit. The findings and the response. **Done when** every finding has an agreed status: fixed, accepted risk or disputed with reasons.